# Machine Learning — Challenge 7
# Transfer Learning: Few-Shot Classification, Neural Style Transfer, and Domain Shift Adaptation

**Prof. Carlos Andrés Sierra, M.Sc.**
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

## Overview

This challenge is a student project on **Transfer Learning** with three parts:

**Part A: Few-Shot Classification $\longrightarrow$ Part B: Neural Style Transfer $\longrightarrow$ Part C: Domain Shift Adaptation**

Part A uses a pretrained backbone to classify new images with only a few examples per class. Part B uses Neural Style Transfer to create stylised images from the target domain. Part C uses those images to adapt the classifier when the source and target domains do not look the same. The task shows how a model trained on *real photographs* behaves when it is tested on *cartoon or stylised images* of the same classes.

The same kind of domain shift appears in autonomous driving (simulators vs. real roads), medical imaging (scanner A vs. scanner B), and satellite analysis (daylight vs. night imagery). Students will measure the shift and compare ways to reduce it.

## Learning objectives

In this challenge, students will:

- *Load and use* pretrained convolutional neural networks (`ResNet-50`, `EfficientNet-B0`, `VGG-19`) from `torchvision.models` or `timm`.

---

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

- *Apply* two transfer learning strategies: **feature extraction** (frozen backbone, only the head is trained) and **fine-tuning** (backbone partially or fully unfrozen) and compare them empirically.
- *Implement* Gatys-style **Neural Style Transfer**: extract content and style representations, define a combined loss, and optimise a target image pixel-by-pixel using `L-BFGS`.
- *Quantify domain shift*: measure the accuracy gap between source-domain test performance and target-domain test performance.
- *Apply domain adaptation* by fine-tuning on a small set of target-domain (cartoon/stylised) images and by using style-transferred images as synthetic target-domain data augmentation.
- *Compare* adaptation strategies on the same domain shift benchmark and justify the best approach with empirical evidence.
- *Communicate* findings in a scientific IEEE paper that covers all three parts and draws a unified conclusion.

## Challenge objective

Each group will complete the work in this order:

1. **Step 1: Choose the dataset pair.** Use the assigned source domain and target domain for the group.
2. **Step 2: Train the baseline classifier.** Train a few-shot image classifier on the source domain with a frozen pretrained backbone, then fine-tune it. Report accuracy on both source-domain and target-domain test splits.
3. **Step 3: Generate stylised images.** Apply Neural Style Transfer to create synthetic images using source content and target style.
4. **Step 4: Adapt the model.** Compare three strategies: no adaptation, fine-tuning with a small labelled target set, and training with the style-transferred images.
5. **Step 5: Compare and report.** Summarise the results, explain the domain shift, and write the final report.

## Dataset assignments (one per group)

All groups use the **`DomainNet`** benchmark dataset (Peng et al., 2019), which contains the same object categories rendered across six visual domains: *Real*, *Clipart*, *Sketch*, *Painting*, *Quickdraw*, and *Infograph*. Each group is assigned a specific **source domain**, a **target domain**, and a **category subset** (6 classes).

1. **Group 1 — Real → Clipart | Animals.** Categories: `bear, bird, cat, dog, horse, rabbit`. Source: DomainNet/real; target: DomainNet/clipart. The clipart domain uses flat colours and simplified contours, creating a sharp texture and detail shift.

2. **Group 2 — Real → Sketch | Vehicles.** Categories: `airplane, bicycle, bus, car, motorcycle, train`. Source: DomainNet/real; target: DomainNet/sketch. Edge-only representations remove all colour and texture cues, making this a strong distribution shift.

3. **Group 3 — Real → Painting | Outdoor Scenes.** Categories: `beach, bridge, forest, mountain, river, tree`. Source: DomainNet/real; target: DomainNet/painting. Paintings retain spatial layout but alter colour distributions and fine-grained textures significantly.

4. **Group 4 — Real → Quickdraw | Household Objects.** Categories: `bed, chair, clock, lamp, sofa, table`. Source: DomainNet/real; target: DomainNet/quickdraw. Quickdraw consists of crowdsourced hand-drawn doodles: the most extreme abstraction in DomainNet.

5. **Group 5 — Real → Infograph | Food.** Categories: `apple, banana, cake, pizza, sandwich, strawberry`. Source: DomainNet/real; target: DomainNet/infograph. Infographics combine flat design, icons, and stylised text labels — a visually composite shift.

6. **Group 6 — Clipart → Sketch | Clothing.** Categories: `jacket, pants, shoe, sock, t-shirt, umbrella`. Source: DomainNet/clipart; target: DomainNet/sketch. This *non-photo* source shifts the problem away from photorealism to test whether transfer learning generalises beyond the real-image prior.

7. **Group 7 — Real**
   *to* **Painting | Office Products.** Dataset: `Office-Home` (Venkateswara et al., 2017), Art
   *to* Clipart split. Categories: `calculator, keyboard, laptop, monitor, mouse, printer`. Office-Home is a dedicated domain adaptation benchmark with four domains and 65 classes; use the 6 listed categories for a tractable experiment.

8. **Group 8 — `SVHN`**
   *to* `MNIST` **| Digit Recognition.** Source domain: `SVHN` (Street View House Numbers — real photographs of digit sequences). Target domain: `MNIST` (handwritten grayscale digits). This is a classic domain shift benchmark: photorealistic, multi-channel, cluttered digits vs. clean, single-channel, handwritten digits. This pair also supports unsupervised domain adaptation baselines (`DANN`).

Download instructions: DomainNet is available at `http://ai.bu.edu/M3SDA/`; Office-Home at `https://hemanthdv.github.io/officehome-dataset/`; SVHN at `http://ufldl.stanford.edu/housenumbers/`; MNIST via `torchvision.datasets.MNIST`.

## Part A — Few-Shot Classification with Transfer Learning

### Motivation

Training a deep CNN from scratch requires tens of thousands of labelled examples per class to avoid overfitting. Transfer learning solves this by initialising the model with weights learned on a large dataset (typically `ImageNet`) and adapting only a small portion of the network to the new task. The intuition: early convolutional layers learn universal edge-and-texture detectors that transfer across domains; only the final task-specific layers need to be re-learned. network to the new task. The intuition: early convolutional layers learn universal edge-and-texture detectors that transfer across domains; only the final task-specific layers need to be re-learned.

### Two strategies to compare

- **Feature extraction (frozen backbone)**: freeze all pretrained weights; replace and train only the classification head (a linear layer or small MLP). Use this as the first baseline. Fast and requires very few labelled examples.
- **Fine-tuning**: after training the head, unfreeze the top 2–3 convolutional blocks and continue training end-to-end with a low learning rate ($10^{-5}$–$10^{-4}$). This adapts higher-level features to the new domain.

Compare both strategies against a **from-scratch** baseline trained only on the few-shot source data, to demonstrate the value of pretraining.
**Suggested few-shot budget**: use only **50 labelled images per class** for training (300 total for 6 classes). Evaluate on the full source-domain test split.

Listing 1: Feature extraction and fine-tuning with ResNet-50 (PyTorch)

```python
import torch
import torchvision.models as models
import torch.nn as nn

# --- Feature extraction (frozen backbone) ---
model = models.resnet50(weights='IMAGENET1K_V2')
for param in model.parameters():
    param.requires_grad = False          # freeze all layers

num_classes = 6
model.fc = nn.Linear(model.fc.in_features, num_classes)  # new head

optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# --- Fine-tuning: unfreeze last two residual blocks ---
for name, param in model.named_parameters():
    if 'layer3' in name or 'layer4' in name or 'fc' in name:
        param.requires_grad = True

optimizer_ft = torch.optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-4
)
```

```python

# --- Training loop (same for both strategies) ---
criterion = nn.CrossEntropyLoss()
for epoch in range(num_epochs):
    model.train()
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

**Preprocessing**: apply the standard `ImageNet` normalisation to all images (mean $[0.485, 0.456, 0.406]$, std $[0.229, 0.224, 0.225]$), resize to $224 \times 224$. Apply random horizontal flip and colour jitter during training as data augmentation.

## Part B — Neural Style Transfer

### Motivation

Neural Style Transfer (Gatys et al., 2015) separates the *content* of an image ("what objects are depicted") from its *style* ("how it is painted") by exploiting the internal representations of a deep pretrained network. Content is encoded in the deep feature maps of a VGG-19 network; style is encoded as the *Gram matrices* of shallower feature maps.

In the context of this challenge, Style Transfer becomes a tool: by transferring the *style* of cartoon/clipart images onto *content* from real photographs, we produce synthetic training images that inhabit the target domain — a form of unsupervised data augmentation for domain adaptation.

### Loss functions

Let $F^l$ denote the feature map at layer $l$ for the *generated* image, $P^l$ for the *content* image, and $G^l$, $\hat{G}^l$ the Gram matrices for the generated and *style* images respectively. The Gram matrix captures pairwise feature correlations:

$$G^l_{ij} = \frac{1}{C_l H_l W_l} \sum_k F^l_{ik} F^l_{jk}, \tag{1}$$

where $C_l$, $H_l$, $W_l$ are the channel, height, and width dimensions at layer $l$.
The total loss is:

$$\mathcal{L}_{\text{total}} = \alpha\, \mathcal{L}_{\text{content}} + \beta\, \mathcal{L}_{\text{style}}, \tag{2}$$

$$\mathcal{L}_{\text{content}} = \frac{1}{2} \sum_{l \in \mathcal{C}} \|F^l - P^l\|_F^2, \qquad \mathcal{L}_{\text{style}} = \sum_{l \in \mathcal{S}} \|G^l - \hat{G}^l\|_F^2. \tag{3}$$

Unlike a standard neural network, here we optimise the *pixel values* of the generated image directly, not model weights.

Listing 2: Gram matrix, losses, and `L-BFGS` optimisation (PyTorch)

```python
import torch
import torch.nn as nn
```

```python
import torchvision.models as models

def gram_matrix(feat):
    b, c, h, w = feat.size()
    feat = feat.view(b, c, h * w)
    return torch.bmm(feat, feat.transpose(1, 2)) / (c * h * w)

class StyleContentLoss(nn.Module):
    def __init__(self, vgg, content_layers, style_layers):
        super().__init__()
        self.vgg = vgg
        self.content_layers = content_layers
        self.style_layers = style_layers

    def forward(self, x, content_target, style_targets):
        content_loss, style_loss = 0.0, 0.0
        for name, layer in self.vgg.features.named_children():
            x = layer(x)
            if name in self.content_layers:
                content_loss += nn.functional.mse_loss(x, content_target[name])
            if name in self.style_layers:
                style_loss += nn.functional.mse_loss(
                    gram_matrix(x), gram_matrix(style_targets[name]))
        return content_loss, style_loss

# --- Optimise image pixels with \texttt{L-BFGS} ---
generated = content_image.clone().requires_grad_(True)
optimizer = torch.optim.LBFGS([generated], lr=1.0, max_iter=20)

alpha, beta = 1.0, 1e4
for step in range(300):
    def closure():
        optimizer.zero_grad()
        c_loss, s_loss = model(generated, content_feats, style_feats)
        loss = alpha * c_loss + beta * s_loss
        loss.backward()
        return loss
    optimizer.step(closure)
```

**Deliverable from Part B**: generate at least **30 style-transferred training images** per class (180 total) by applying the style of target-domain images onto real-photo content images. These will be used as synthetic data in Part C.
**Suggested layers**:

- Content layer: `relu4_2` (VGG-19 block 4, second conv).
- Style layers: `relu1_1`, `relu2_1`, `relu3_1`, `relu4_1`, `relu5_1` (one from each block).

## Part C — Domain Shift Measurement and Adaptation

### Measuring domain shift

After training the best Part A classifier on the source domain, evaluate it directly on the *target domain* test split **without any adaptation**. The gap between source-test

accuracy and target-test accuracy is the **domain shift penalty**:

$$\Delta_{\text{shift}} = \text{Acc}_{\text{source}} - \text{Acc}_{\text{target}}. \tag{4}$$

Report $\Delta_{\text{shift}}$ for both the frozen-backbone and fine-tuned models. Discuss which layers are most responsible for the drop (use feature visualisation or Grad-CAM to inspect what the model attends to in real vs. cartoon images).

**Three adaptation strategies to compare**

1. **Baseline (no adaptation):** the Part A best classifier evaluated directly on target-domain images. Establishes the lower bound.
2. **Target fine-tuning:** collect a small labelled target-domain set (50 images per class) and fine-tune the last convolutional block and head for 10–20 epochs with early stopping. This simulates having a small annotation budget in the target domain.
3. **Style-transfer augmentation:** augment the original 50-per-class source training set with the 30-per-class style-transferred images from Part B (no additional target-domain labels required). Retrain the model and evaluate on the target-domain test split.

**Optional (for groups seeking distinction)**: implement **Domain-Adversarial Neural Networks** (DANN; Ganin et al., 2016). Add a gradient reversal layer and a domain classifier head so the feature extractor is trained to be *domain-invariant*:

Listing 3: Gradient reversal for domain-adversarial training (DANN)

```python
from torch.autograd import Function

class GradientReversal(Function):
    @staticmethod
    def forward(ctx, x, alpha):
        ctx.save_for_backward(torch.tensor(alpha))
        return x.clone()

    @staticmethod
    def backward(ctx, grad_output):
        alpha, = ctx.saved_tensors
        return -alpha * grad_output, None

class DANNClassifier(nn.Module):
    def __init__(self, backbone, num_classes):
        super().__init__()
        self.backbone      = backbone
        self.class_head    = nn.Linear(backbone.fc.in_features,
            num_classes)
        self.domain_head   = nn.Sequential(
            nn.Linear(backbone.fc.in_features, 256),
            nn.ReLU(),
            nn.Linear(256, 2)   # source / target
        )
        backbone.fc = nn.Identity()  # remove original head
```

```python

    def forward(self, x, alpha=1.0):
        feat    = self.backbone(x)
        cls_out = self.class_head(feat)
        rev     = GradientReversal.apply(feat, alpha)
        dom_out = self.domain_head(rev)
        return cls_out, dom_out

# --- Combined loss ---
cls_loss = criterion(cls_out[src_mask], labels[src_mask])
dom_loss = criterion(dom_out, domain_labels)   # 0=source, 1=target
loss     = cls_loss + lambda_d * dom_loss
```

### Suggested workflow

1. **1. Prepare the data.** Download the assigned source and target domains. Build `ImageFolder` or custom `Dataset` classes. Split the source data into 50 train / 50 validation / remaining test images per class. Keep the full target-domain test split for evaluation.

2. **2. Train the source model.** Load `ResNet-50` with `ImageNet` weights, freeze the backbone, replace `fc` with a 6-way linear head, and train for 20–30 epochs.

3. **3. Fine-tune the source model.** Unfreeze `layer3` and `layer4`, continue training with a low learning rate, and evaluate the model on source and target test sets. Record $\Delta_{\text{shift}}$.

4. **4. Create stylised images.** Select one style image per class from the target domain and generate 30 stylised images per class with `L-BFGS`. Save them in `data/synthetic_target/`.

5. **5. Train the adaptation models.** Compare no adaptation, target-domain fine-tuning, and style-transfer augmentation. Evaluate all three on the target test set and report per-class accuracy.

6. **6. Compare the results.** Build one summary table with all model variants, then add Grad-CAM and t-SNE plots to show the domain shift and the effect of adaptation.

7. **7. Write the report.** Explain the source domain, target domain, training setup, results, and the best adaptation strategy.

### Suggested hyperparameter search space

Groups must justify their choices rather than merely copying defaults:

- **Backbone**: `ResNet-50`, `EfficientNet-B0`, `MobileNet-V3` (choose one; compare optionally).

- **Head architecture**: single linear layer vs. two-layer MLP (512 hidden units, `ReLU`, 0.3 dropout).

- **Learning rate**: feature extraction: $\{10^{-3}, 5 \times 10^{-4}\}$; fine-tuning: $\{10^{-4}, 5 \times 10^{-5}\}$.
- **Batch size**: 32; 64.
- **Epochs**: feature extraction: 20–30; fine-tuning: 30–50.
- **Style transfer**: $\alpha/\beta$ ratio $\in \{10^{-3}, 10^{-4}, 10^{-5}\}$, number of optimisation steps $\in \{200, 300, 500\}$.
- **DANN** (optional): $\lambda_d \in \{0.1, 0.5, 1.0\}$; linear $\alpha$ schedule from 0 to 1 over training.

### Evaluation criteria

Submissions will be evaluated on the following aspects:

- **Classification and transfer learning (25%):** correct implementation of feature extraction and fine-tuning; comparison with from-scratch baseline; source-domain accuracy and $\Delta_{\text{shift}}$ reported correctly.
- **Neural Style Transfer (20%):** loss implementation is correct (Gram matrices, content + style terms); stylised images are visually plausible and span representative style variation; at least 30 per class produced.
- **Domain adaptation analysis (30%):** all three adaptation strategies are correctly implemented and compared; performance gap is quantified per class; Grad-CAM or t-SNE visualisations are included and interpreted.
- **Scientific writing and synthesis (15%):** the paper connects the three parts into a coherent narrative; domain shift is explained in terms of the learned feature representations; conclusions are supported by evidence.
- **Code quality, reproducibility, and video (10%):** seeds are fixed, environment is specified, a demonstration video is included.

### Expectations and experimental protocol

Groups must follow a rigorous protocol. At minimum, the following elements are required:

1. Use **identical preprocessing** (same resize, normalisation, augmentations) for source and target domain images to ensure the domain shift is not artefactual.
2. Train all Part A and Part C classification models with at least **three independent random seeds**; report mean and standard deviation of accuracy on both test splits.
3. Report the **domain shift penalty** $\Delta_{\text{shift}}$ for each model variant as a primary result.
4. Produce a **summary comparison table** with all five model variants, source-domain accuracy, target-domain accuracy, and $\Delta_{\text{shift}}$.
5. Include at least **four required figures**:

a. Training and validation loss / accuracy curves for Part A models.

b. Side-by-side gallery: content image, style image, and generated style-transferred image (one example per class, 6 total).

c. Grad-CAM attention maps for one correct and one incorrect prediction per domain (source vs. target).

d. t-SNE projection of Part A backbone features for source and target domain images, coloured by class, shown before and after domain adaptation.

6. Discuss at least one **qualitative case study**: pick two examples where the unadapted model fails on target-domain images and the adapted model succeeds. Explain what visual cues drove the failure and the recovery.

**Common pitfalls (teaching moments)**

- *Forgetting to normalise target-domain images*: the `ImageNet` statistics used to normalise source images may not match the target domain (e.g., cartoon images have different brightness distributions). Regardless, always apply the same `ImageNet` normalisation parameters to both domains — the model expects them.

- *Fine-tuning with a high learning rate on a small dataset*: unfreezing the entire backbone with $\mathrm{lr} = 10^{-3}$ will quickly destroy the pretrained features. Always use a small learning rate ($\leq 10^{-4}$) for the backbone and optionally a larger rate only for the new head.

- *Confusing content and style*: the content layer should be deep in the network (`relu4_2`) to capture object structure; style layers should span all depths to capture textures at multiple scales.

- *Style transfer as a lossless operation*: the generated image is an approximation. High $\alpha/\beta$ ratios preserve content but apply little style; low ratios paint style strongly but may distort content beyond recognition. The right balance depends on the domain gap.

- *Evaluating on the target domain before deciding on the adaptation strategy*: this is data leakage. Fix the adaptation method and all hyperparameters using a target-domain *validation* split; use the test split only for final reporting.

- *Ignoring class imbalance*: DomainNet categories are balanced within each domain split, but the Quickdraw and Infograph splits may contain fewer examples for some classes. Report per-class accuracy, not just macro accuracy.

- *t-SNE of raw pixels vs. features*: visualising raw pixel space typically shows domain separation that is artefactual (image statistics differ). Visualise the *penultimate feature layer* of the backbone to show meaningful semantic structure.

### Deliverables

- A public **GitHub repository** named `challenge-7_<groupid>` containing:
  - *Reproducible code*: separate modules `classifier.py`, `style_transfer.py`, and `domain_adaptation.py` plus Jupyter notebooks for each of the three parts.
  - *Environment specification*: `requirements.txt` or `pyproject.toml`; a `Dockerfile` is strongly encouraged.
  - Exact commands or a `Makefile` to reproduce all reported results. Record all random seeds.
  - *Saved model weights*: `.pt` checkpoints for the best Part A and Part C models.
  - *Synthetic image gallery*: a `data/synthetic_target/` folder with the 180+ style-transferred training images.
  - *Figures*: all four required figures plus any additional ones saved as `.png` or `.pdf`.
- A **scientific paper** in IEEE format, maximum **seven (7) pages** excluding references. The paper must cover all three parts; include abstract, brief related work on transfer learning and domain adaptation, data and setup description, methodology (Parts A, B, C), experimental results with the summary comparison table, qualitative case study, and conclusions.
- A **short video** (7–10 minutes) uploaded to a public or unlisted link (e.g., OneDrive) demonstrating the style transfer results visually and the accuracy improvements from domain adaptation.

### Notes on scope and computational budget

- ResNet-50 feature extraction on 300 images for 30 epochs runs in under 5 minutes on CPU. Fine-tuning requires a GPU for practical speeds but can run on a free-tier Google Colab T4 instance within 30–60 minutes.
- Neural Style Transfer with `L-BFGS` on a single $256 \times 256$ image takes 1–3 minutes on CPU (300 steps). Generating 180 images will therefore take 3–9 hours on CPU; use a GPU or run batches overnight. `fast_neural_style` (Johnson et al., 2016) is a faster feed-forward alternative if time is limited — implement it as an optional comparison.
- For Group 8 (SVHN $\rightarrow$ MNIST), note that MNIST is greyscale; convert SVHN to greyscale or work in 3-channel space by replicating the greyscale channel.

### Reproducibility checklist

- Fix and record all `random_state` / `seed` values (Python, NumPy, PyTorch).
- Provide environment specification (`requirements.txt` or `pyproject.toml`).
- Include run scripts or `Makefile` instructions.
- Confirm identical preprocessing for source and target domains.
- Document the exact DomainNet / Office-Home split used (train/val/test indices or the official split file).

### Grading checklist (to include in the repository)

Each submission must include a short `CHECKLIST.md` containing:

- The assigned dataset pair and category subset.
- The pretrained backbone chosen and the number of trainable parameters in each model variant.
- Source-domain accuracy and target-domain accuracy (mean ± std over three seeds) for all five model variants.
- The domain shift penalty $\Delta_{\text{shift}}$ for the best Part A model and for the best Part C adaptation model.
- The $\alpha/\beta$ ratio used in Neural Style Transfer and a visual quality assessment of the generated images.
- A short paragraph (max 200 words) explaining which adaptation strategy worked best for this group's domain pair and why.

### References and further reading

Include canonical references in the IEEE paper: the original Neural Style Transfer paper (Gatys, Ecker & Bethge, 2016), the fast feedforward alternative (Johnson et al., 2016), the DomainNet benchmark (Peng et al., 2019), DANN (Ganin et al., 2016), the Office-Home dataset (Venkateswara et al., 2017), and transfer learning surveys (Tan et al., 2018; Zhuang et al., 2021). Cite the pretrained model architecture used (`ResNet`: He et al., 2016; `EfficientNet`: Tan & Le, 2019). All citations must be in IEEE style.

*Challenge 7 ties together the computational power of deep pretrained representations and the scientific problem of distribution shift. The ability to reuse knowledge across domains — whether from `ImageNet` to cartoons, from photographs to paintings, or from simulators to the real world — is one of the most practically consequential skills in modern machine learning.*